



PEJU/AND/ADA/APP WEB IA

DESARROLLO DE APLICACIONES WEB CON IA

M.3011.002.083

Desarrollo Backend - Node

Juan Manuel Castillo Andrades



Cofinanciado por
la Unión Europea



Fondos Europeos



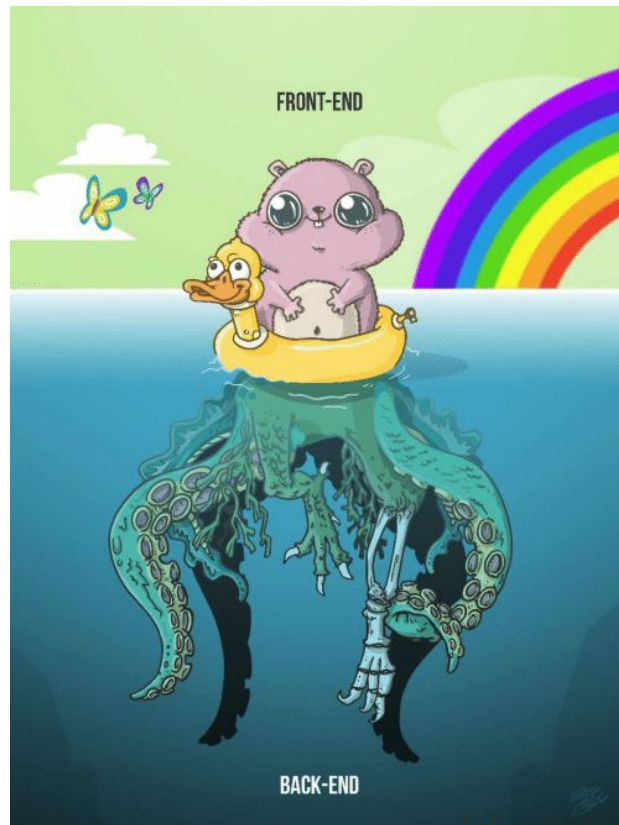
Sistema Nacional de
Garantía Juvenil





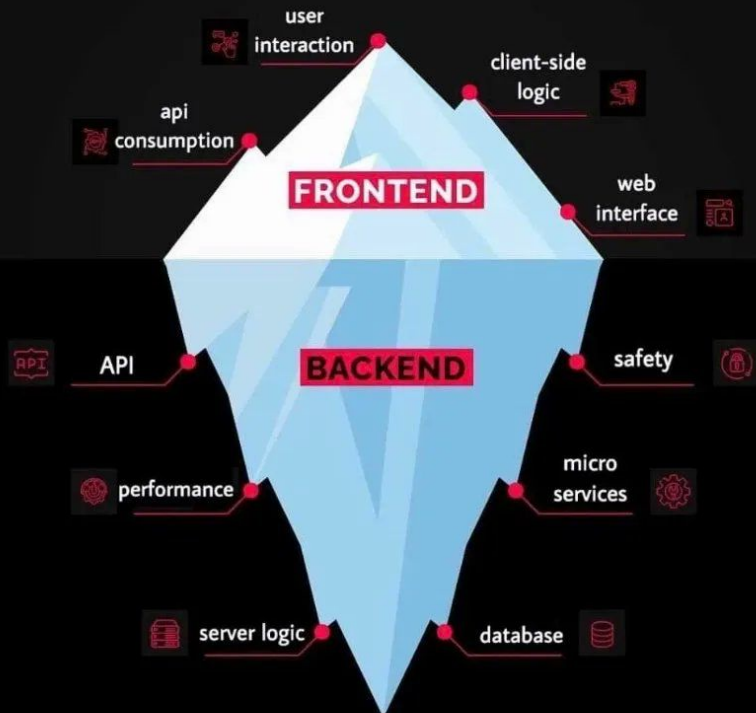


¿Dónde nos adentramos?



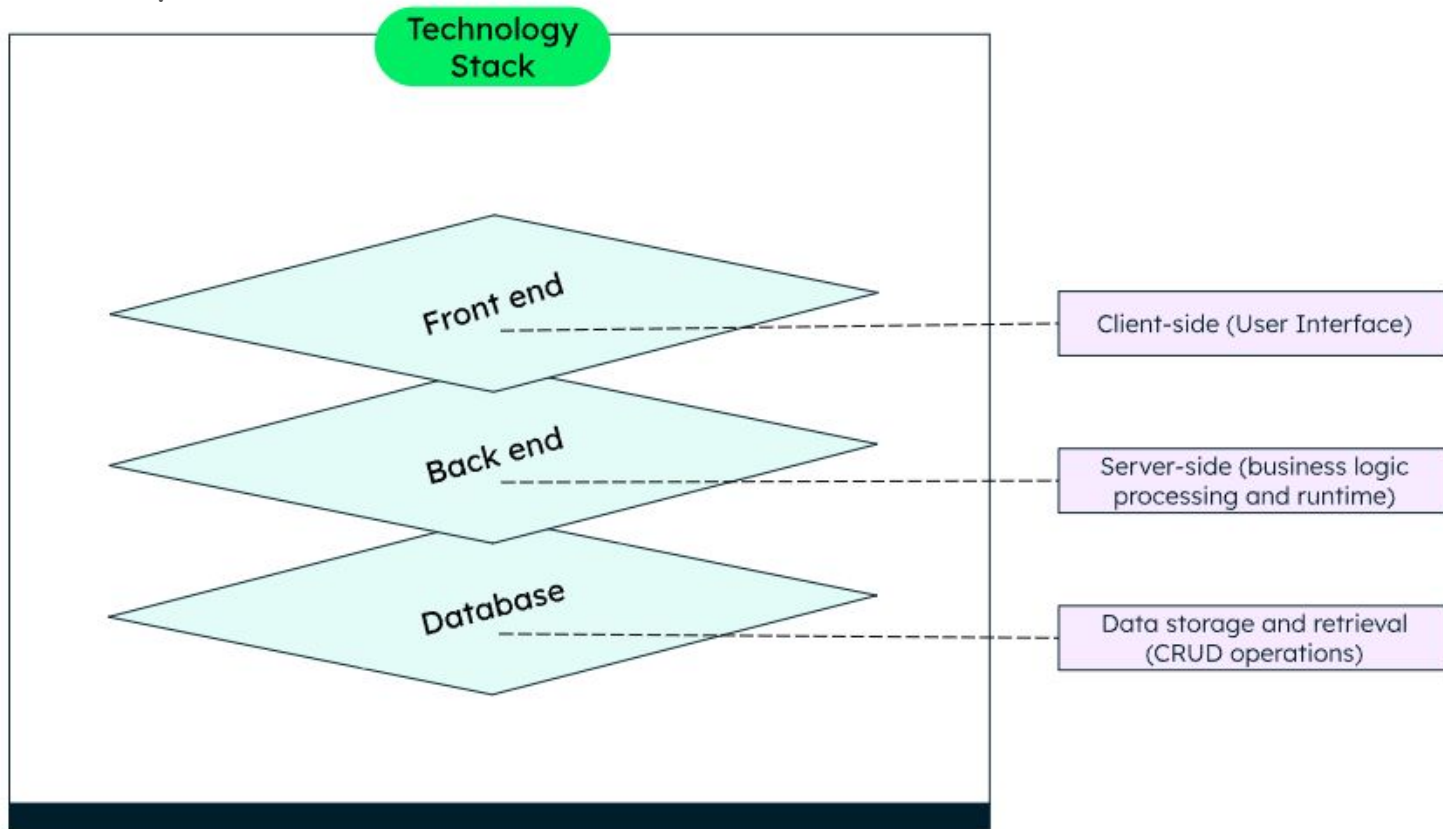


FRONT-END | BACK-END



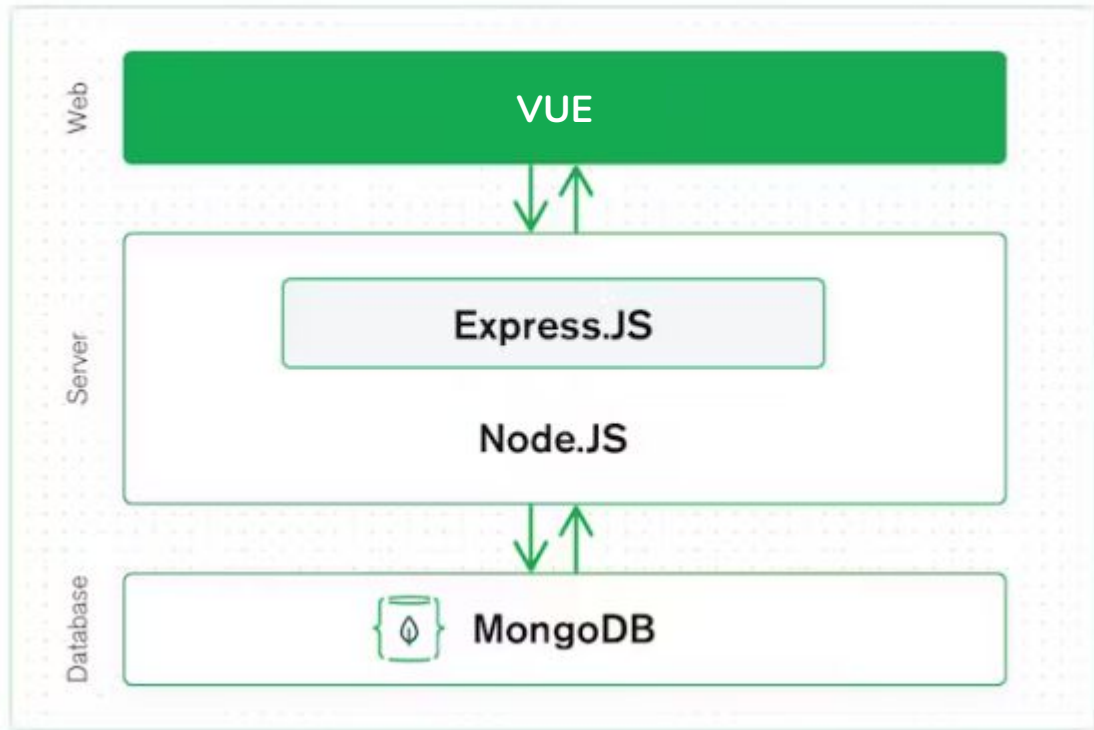


Capas en una aplicación





Nuestro Stack



MEVN

- ◆ Mongo
- ◆ Express
- ◆ Vue
- ◆ Node



MERN

- ◆ Mongo
- ◆ Express
- ◆ React
- ◆ Node



MEAN

- ◆ Mongo
- ◆ Express
- ◆ Angular
- ◆ Node



Frontend - Contexto





Backend - Contexto





El mayor problema de backend...





El mayor problema de backend...

La abstracción





Take it easy



El truco...





“Entender el hilo de ejecución de nuestra aplicación”







¿Qué es?

- Entorno de programación basado en **JavaScript** (creado en 2009)
- En el contexto del **lado del servidor**
- Muy unido a **persistencia NoSQL** (MongoDB)
- Usa el **motor interno V8** (motor de Google Chrome)
- **Alto rendimiento** (desarrollado en C/C++ 80%)
- Hay una **gran comunidad** por detrás / muy vivo
- Es una **marca registrada** pero **Open Source**



¿Qué es?

*“Node es una **entorno** construido sobre el motor JavaScript del navegador Google Chrome para crear fácilmente **aplicaciones rápidas y escalables**”*



¿Qué es?

¡OJO! Node *no es un framework o Lenguaje*, es un **entorno** de JavaScript, pero tiene todo lo que un lenguaje de programación necesita.



¿Qué es?

Características de un lenguaje de programación

- **Una Gramática** que define su sintaxis
- **Un compilador** que lo interpreta y ejecuta
- **Mecanismos** para **interactuar con el exterior**
- **Librerías** estándar (consola, ficheros, red, etc.)
- **Utilidades** (intérprete, depurador, gestor de paquetes, etc.)



¿Qué es?

Características de un lenguaje de programación

V8

- **Una Gramática** que define su sintaxis
- **Un compilador** que lo interpreta y ejecuta

Node

- **Mecanismos para interactuar con el exterior (con el contexto de ejecución)**
- **Librerías (módulos)** estándar (terminal, sistema de ficheros, red, etc.)
- **Utilidades** (intérprete, depurador, paquetes, etc.)



Arquitectura

- Basada en eventos (Asincronía)
- No bloqueante (aunque podemos usar un enfoque bloqueante)
- De entradas/salidas (I/O)
- Ligero y eficiente (especialmente en aplicaciones en tiempo real)



Arquitectura tradicional vs Node

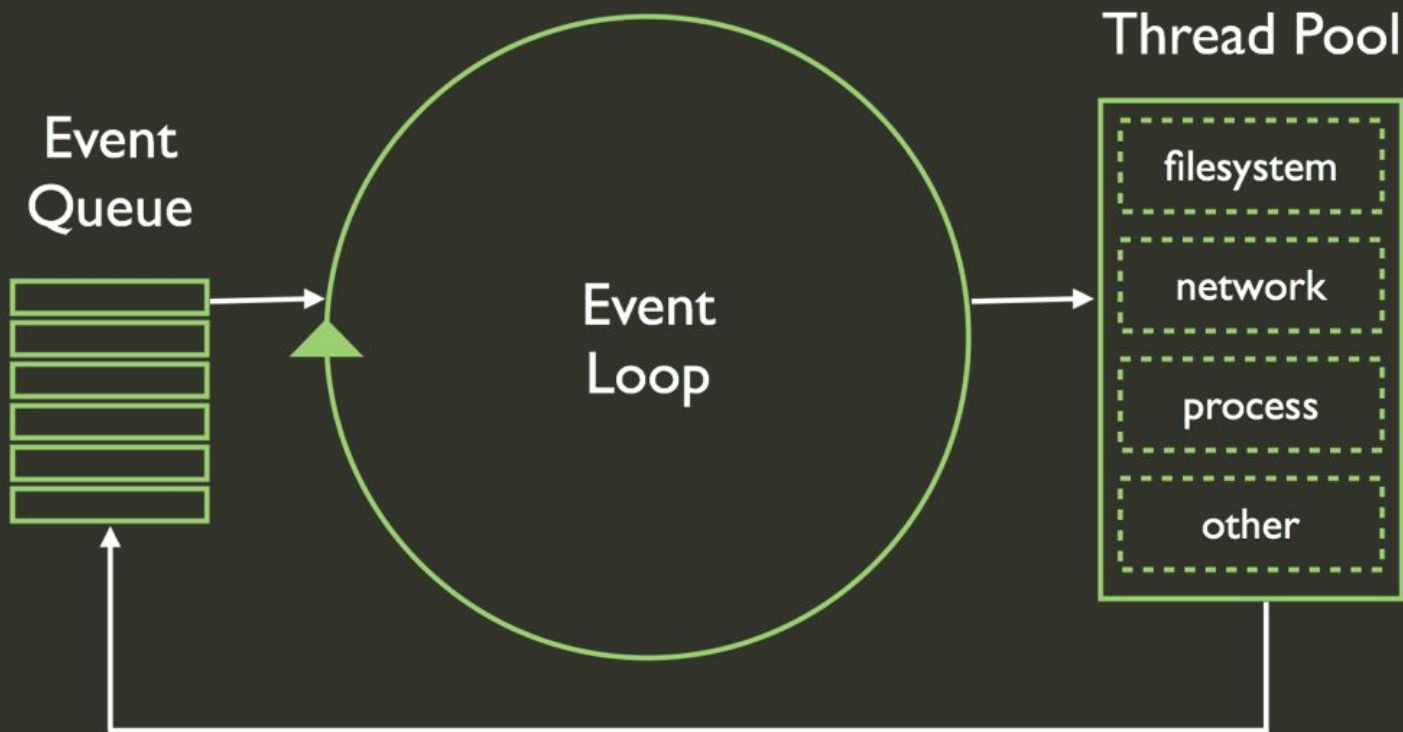




Node Event loop

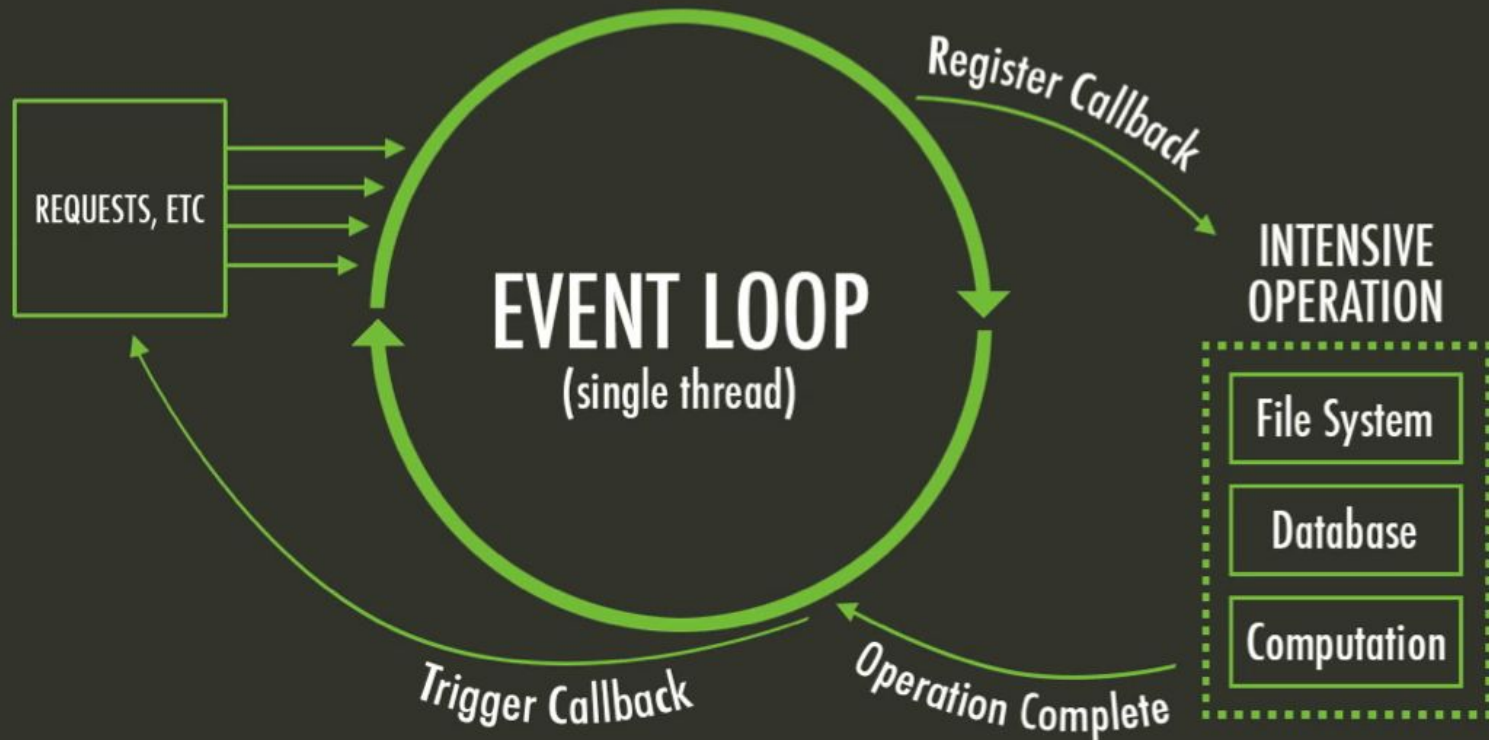


Event Loop



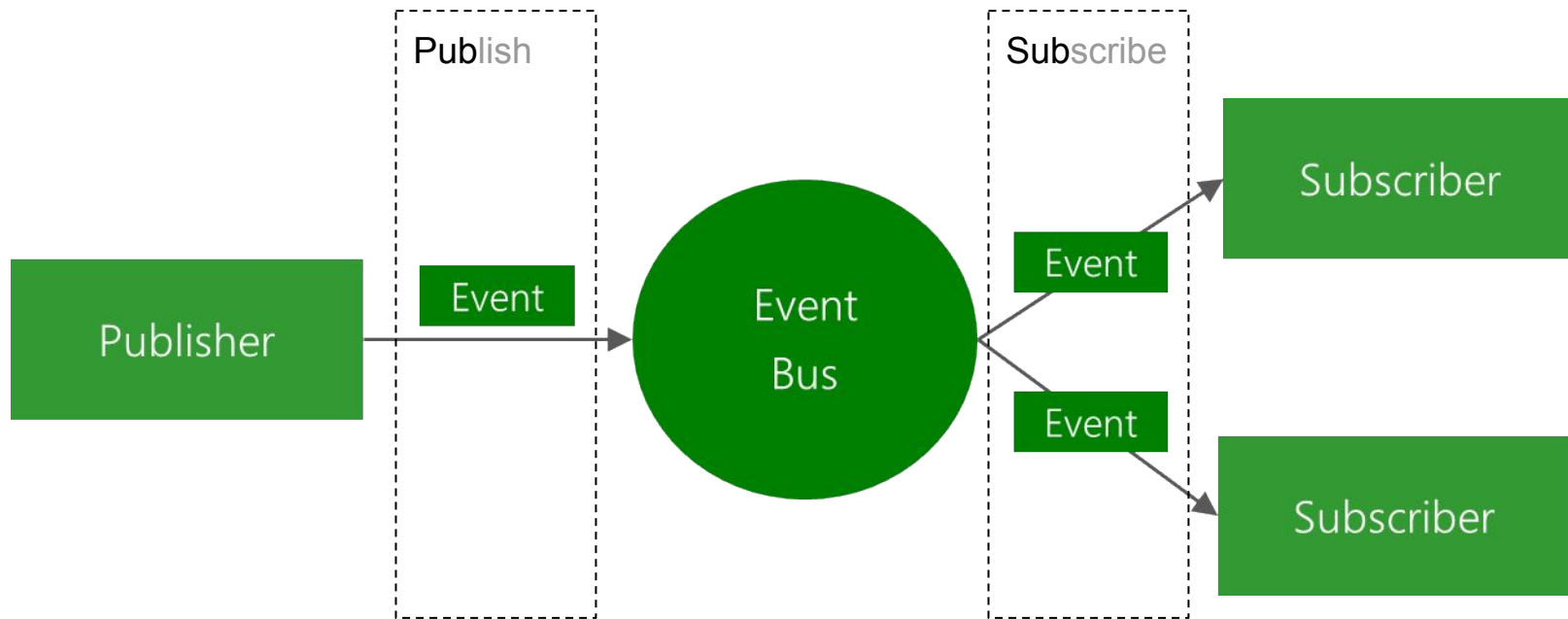


Event Loop



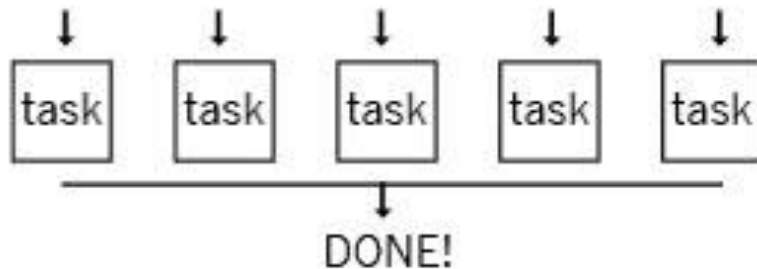
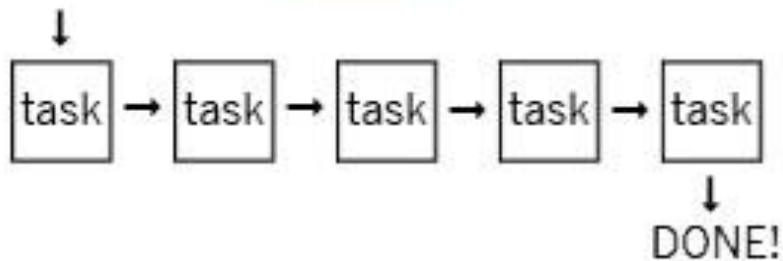


Patrón Pub / Sub





Bloqueante vs no bloqueante





Bloqueante vs no bloqueante

*Aunque Node esté enfocado a ejecución no bloqueante, **NO limita** por lo que podemos plantear **nuestro código** de manera **bloqueante o no bloqueante**.*



NO BLOQUEANTE != ASÍNCRONO

BLOQUEANTE != SÍNCRONO



Bloqueante vs no bloqueante

Ejemplo lectura de fichero en sistemas bloqueantes y no bloqueantes.

BLOQUEANTE	NO BLOQUEANTE
Abrir el archivo	Abrir el archivo
Leer el archivo	Leer el archivo
Pintar el contenido	Pintar el contenido (cuando termines)
Hacer algo más	Hacer algo más

Ejemplo Node - Bloqueante

```
import { readFileSync } from 'node:fs';

console.log('\nAbriendo Archivo...');

let content = readFileSync('eoi.txt', 'utf8');
console.log(content);

console.log('\nHacemos otra cosa');
```

Ejemplo Node - No bloqueante

```
import { readFile } from 'node:fs';

console.log('\nAbriendo fichero...');

readFile('eoi.txt', 'utf8', function (error, content) {
  console.log(content);
});

console.log('\nHacemos otra cosa');
```



¿Por qué Node?

- Es tu **mejor opción** para programar del lado backend porque **ya sabes JavaScript**
- Si no te gusta el backend, sus módulos te harán mejor frontend
- Podéis **comenzar** vuestra **carrera profesional** como **Full Stack JavaScript Developer**



¿Qué podemos hacer con Node?

- Aplicaciones Web ([PWA's](#))
- Páginas Web, Blogs, CMF's, CMS's
- Servicios en la nube (API's con infraestructuras [headless](#))
- Chats, juegos, clientes de correo, traductores, plugins para VS Code
- Sistemas de estadísticas y analíticas... [Big Data](#)
- Aplicaciones de Red (línea de comandos o administración de sistemas)
- Controlar [Hardware](#), por ejemplo [drones](#)
- [Machine learning](#), [Tensorflow](#).

ALL CLEAR



ALL CLEAR ?





Node Instalación



Instalación



Instalar versión de Node con NVM

```
// instalando versiones
```

```
nvm install 22
```

```
nvm install 22.0.0
```

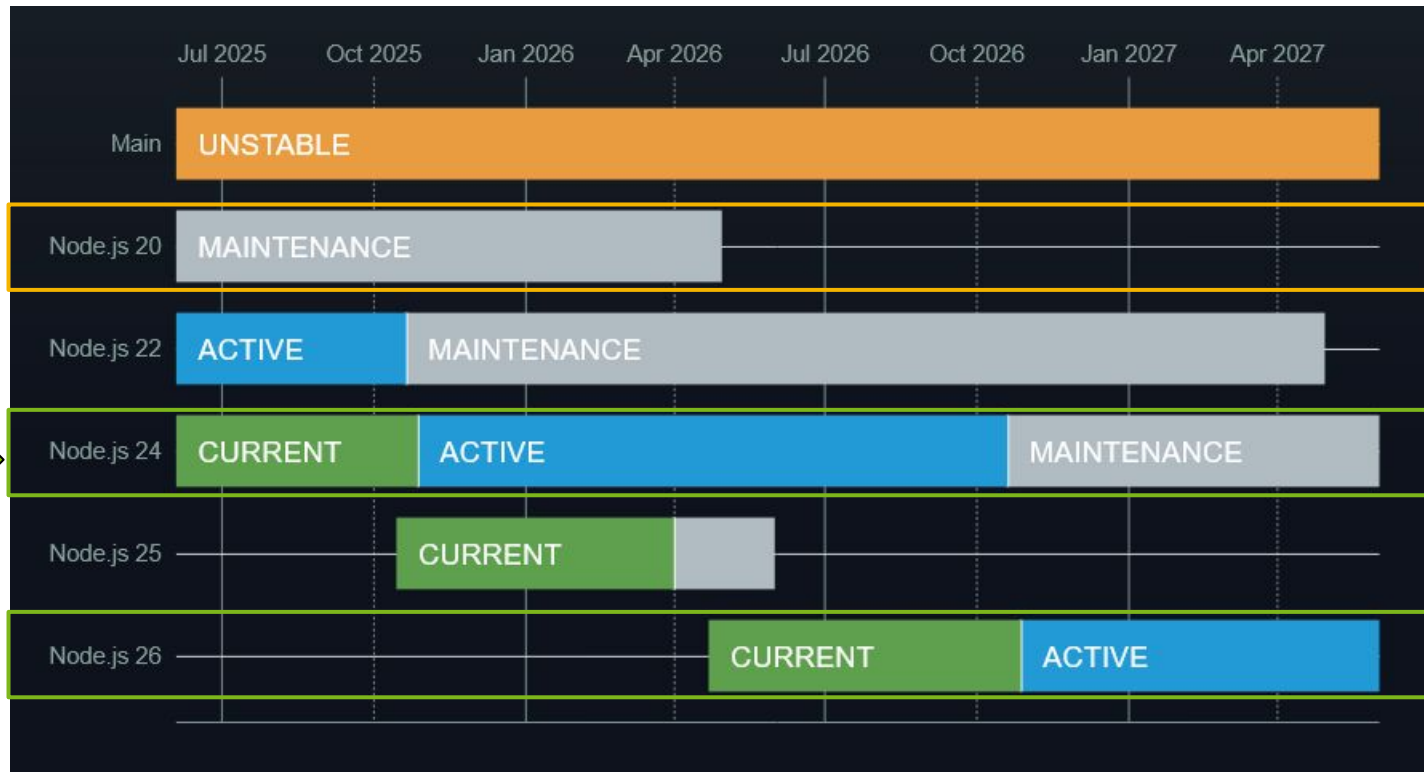
```
// cambiar de versión
```

```
nvm use 22
```

```
nvm use 20
```



¿Qué versión instalar?



Comprobar versiones instaladas

```
// versión actual de node
```

```
node -v
```

```
// versión actual de npm
```

```
npm -v
```

Comprobar versiones instaladas

```
// versión actual de node
```

```
node -v
```

```
output: v24.15.0
```

```
// versión actual de npm
```

```
npm -v
```

```
output: 11.12.1
```



Node

Principales módulos



Módulos

Algunos de los módulos que vienen con Node y que podemos usar:

Módulos	
<u>Globals</u>	<u>Process</u>
* <u>HTTP / HTTPS</u>	<u>Buffer</u>
<u>Stream</u>	* <u>File System</u>
* <u>Events</u>	<u>Util</u>
<u>Timers</u>	<u>Modules</u>



Pero antes de usarlos...
... aprendamos cómo gestionarlos



MÓDULOS

Estabilidad y versionado



Gestión de módulos

Es importante revisar su nivel de estabilidad al usarlos y entender la numeración de las versiones.

Nivel de estabilidad	Descripción
Stability: 0 - Deprecated	Emitirá avisos de desaparición en el futuro. La retrocompatibilidad no está garantizada.
Stability: 1 - Experimental Stability: 1.0 - Early development Stability: 1.1 - Active development Stability: 1.2 - Release Candidate	Esta característica aún se encuentra en desarrollo activo y está sujeta a cambios o eliminaciones no compatibles con versiones anteriores en cualquier versión futura. El uso de la función no se recomienda en entornos de producción. Las características experimentales no están sujetas a semantic versioning.
Stability: 2 - Stable	Compatibilidad con el ecosistema npm, versiones retrocompatibles dentro de semantic versioning.
Stability: 3 - Legacy	Funcionalidades que no se quieren eliminar, seguirán presentes en las próximas versiones pero no hay desarrollo activo y existirán alternativas.



Gestión de módulos

Ejemplos según nivel de estabilidad

Nivel de estabilidad	Descripción
Stability: 0 - Deprecated	<u>NET > server.connections</u>
Stability: 1 - Experimental	<u>CorePack</u>
Stability: 2 - Stable	<u>Crypto</u>
Stability: 3 - Legacy	<u>Event.cancelBubble</u>

El **nivel de estabilidad** puede referirse a un **módulo al completo** o a **métodos concretos**.



Node

Organizando el código



Node

Modules

Importar contenido

Importar código de otros ficheros ES6 syntax

```
import fs from 'node:fs' //trae todo el contenido exportado en node:fs
import {readFileSync} from 'node:fs' //trae un elemento en concreto exportado en node:fs
import database from './database.js' //trae el contenido exportado en el fichero database (mismo directorio)
import explorer from '../explorer.js' //trae el contenido exportado en el fichero explorer (directorio padre)
```




Node Modules Exportar contenido

Exportar código para ser usado en otros ficheros (varios elementos)

```
//Inicio fichero <example1.js>
console.log("evaluating example.js")

let invisible = function () {
  console.log("invisible");
}

export const message = "hi"

export const say = function () {
  console.log(message)
}

//Fin fichero <example1.js>
```

Exportar código para ser usado en otros ficheros (un único elemento)

```
//Inicio fichero <example2.js>  
console.log("evaluating example.js")
```

```
let invisible = function () {  
  console.log("invisible")  
}
```

```
const message = "hi"  
const say = () => {  
  console.log(message)  
}
```

```
export default {message, say}  
//Fin fichero <example2.js>
```

Uso del código en example.js

```
//importa el contenido de example.js (mismo directorio)
import {say} from './example1.js'
import example from './example2.js'

//envía por la salida estándar (CLI) el contenido exportado
console.info(example)

//llama al método say directamente desde la variable declarada example
example.say()

//llama al método say contenido en example
say()
```



MÓDULO FILESYSTEM

Acceso al sistema de ficheros

Módulo FileSystem - Escribiendo un fichero

```
import { writeFileSync } from 'node:fs';  
  
// Si el fichero existe, sobrescribe el contenido  
writeFileSync("hola-mundo.txt", "¡Hola mundo!");  
  
// Crea el fichero en un subdirectorio files  
writeFileSync("files/hola-mundo.txt", "¡Hola mundo!");
```

Módulo FileSystem - Añadiendo contenido a un fichero

```
import { appendFileSync } from 'node:fs';  
  
// si el fichero no existe, se crea  
appendFileSync("hola-mundo.txt", "\n¡Hola mundo!")
```

Módulo FileSystem - Leyendo un fichero

```
import { readFileSync } from 'node:fs';  
  
let data = readFileSync("hola-mundo.txt", "utf8");  
  
console.log(data);
```


Módulo FileSystem - Borrando un fichero

```
import { unlinkSync } from 'node:fs';
```

```
unlinkSync("hola-mundo.txt");
```

Práctica. Manejando ficheros

Instrucciones

Crear 4 scripts que trabajen sobre el fichero en **files/hola-mundo.txt**:

Script

Descripción

create-file.js

Crea con el texto “Hola mundo”

read-file.js

Lee contenido actual

append-file.js

Añade “Hola mundo” al contenido existente

delete-file.js

Borra el fichero

Notas

Si se realiza una acción sobre un fichero que no existe o tiene alguna limitación debe manejar las excepciones que se puedan ocurrir y terminar la ejecución de manera controlada.